Team13 project report

Dahyun Kang [1] **Subin Kim** [2] **Soohyeong Kim** [3] **Joonseo Jang** [1]

Abstract

We applied various LLM-based approaches to solve a simplified version of the ARC-AGI task. First, we augmented the dataset through topological transformations and Python-based procedural generation. Next, we fine-tuned a quantized LLaMA model, including its embedding layers. To handle the data diversity, we employed a Mixture-of-Experts (MoE) setup by dividing the dataset into seven classes and assigning a distinct adapter to each class. During inference, we introduced data augmentation at test time and adjusted the model's guesses based on confidence levels. We also experimented with Test Time Training (TTT) to further improve performance. The final model was selected based on an optimal combination of MoE adapter configurations, DFS search parameters, and the degree of TTT applied.

1. Introduction

Large Language Models (LLMs) have demonstrated impressive capabilities across a wide range of tasks, from natural language processing to code generation. However, the core question of whether these systems can truly reason remains an open debate within the artificial intelligence community. The Abstraction and Reasoning Corpus (ARC-AGI) (Chollet, 2019), specifically designed to evaluate genuine reasoning in AI systems, exemplifies this challenge. In this work, we propose a method to solve a simplified version of the ARC-AGI task, where each instance is represented as a 10×10 grid. The model was trained with limited GPU resources. Our approach achieved 73% accuracy on the official evaluation server, demonstrating the effectiveness of the proposed system.

All authors contributed equally to this work. [1]Department of Computer Science and Engineering [2]Department of Physics and Astronomy [3]Department of Mathmetics.

Copyright 2025 by the author(s).

2. Overview

The diagram below illustrates the overall structure and training process of our proposed model:

- Initially, the base model undergoes fine-tuning using an augmented data set.
- A separate MoE classifier is then used to classify all training examples into distinct categories. Each subset is subsequently used to fine-tune a different MoE adapter.
- During inference, the evaluation dataset is augmented to generate multiple candidate outputs. These candidates are passed through the appropriate MoE path, individually scored, and the best prediction is selected based on the highest score.

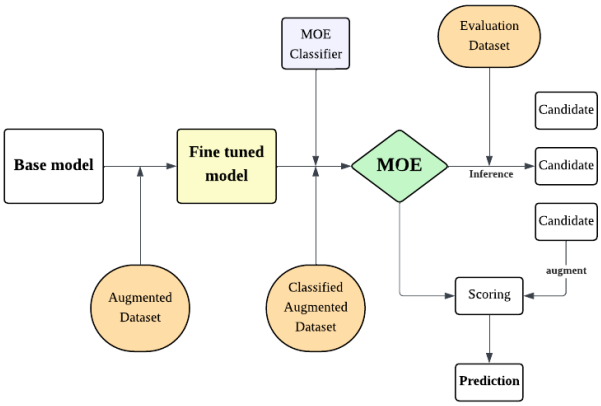

Figure 1. overview of full process of model

3. Dataset

To train our model, we constructed a new dataset based on the original data provided in the course.

3.1. Usual augmentation

We first applied basic data augmentation techniques—such as rotation, reflection, and color transformation—to the original dataset. During this process, we visually inspected all

task sets and manually selected only those transformations that preserved the original labels. This procedure was implemented using a custom script, ‘data_augmentation.py‘. In addition, to address task-level data imbalance, we examined the number of examples per task. For subsequent MoE training, we used only the tasks with a sufficient number of examples.

3.2. Data Generation

In addition to data augmentation, we generated entirely new tasks using the GPT-4o-mini model via its API, following the exact procedure described by Wen-Ding Li *et al.* (Li et al., 2024), in *Combining Induction and Transduction for Abstract Reasoning*. Specifically, we adopted their approach to:

- Extract problem seeds from existing ARC tasks.
- Prompt GPT-4o-mini to produce Python functions implementing these seeds.
- Use the generated functions to create synthetic input-output pairs for training.

This process mirrors Li’s pipeline, where GPT-4/GPT-4o-mini synthesize variations of ARC tasks and program-based solutions to expand the training dataset to around 400K examples. By integrating this method, we enhanced both the diversity and quantity of training data, enabling more robust learning in our MoE framework. We generated 295 tasks with more than 500 examples for each.

4. Base model

We used LLaMA-3.2-3B-Instruct as the base model for training. LLaMA 3 is a multilingual, text-only language model released by Meta. (Meta AI, 2024) We selected this model because, after quantization, it supports efficient fine-tuning and inference within the constraints of a limited GPU environment.

5. Training strategy

we are using two types of training strategies, which are a usual fine tuning, and Mixture of Experts(MoE).

5.1. pretraining

Due to resource constraints, we adopted Low-Rank Adaptation (LoRA) for fine-tuning rather than training the model from scratch. Our overall training strategy was inspired by the implementation of Daniel Franzen et al., the winner of the 2024 ARC-AGI competition. We applied LoRA to both the self-attention and feed-forward layers of the Transformer blocks. To reduce the embedding dimension, we also included the ‘embed_tokens‘ and ‘lm_head‘ modules in the adaptation process. Furthermore, considering the nature of the ARC-AGI task, we replaced the base model’s original embedding tokens with a reduced token set. This design choice also follows the methodology proposed by Franzen et al. (Franzen et al., 2024) We experimented with LoRA ranks of 64 and 256 and observed no significant difference in performance on external evaluation datasets. Therefore, we selected rank 64 for the final model to reduce training time. For efficient training of the quantized model, we utilized the Unsloth library. For initial training, however, we did not use the above adaptation strategy and instead trained on the full set of 70,000 original examples to reduce training time. We used a linear decay scheduler with a warmup ratio of 0.1. This choice was based on research findings showing its effectiveness under similar conditions when fine-tuning LLMs with LoRA . (Bergsma et al., 2025) When compared under identical experimental conditions to the scheduler proposed by Daniel Franzen et al., our scheduler demonstrated faster convergence and higher accuracy. The main hyperparameters used for initial training are summarized below.

Table 1. Each hyperparameters we used for two types of training.

HYEPRPARAMETER	PRETRAINING	MOE
EPOCHS	1	1
BATCH SIZE	4	4
GRAD_ACM_STEPS	2	2
LORA RANK	64	64
LORA ALPHA	32	32
LR (LORA ADAPTERS)	1E-4	3E-5
LR (EMBEDDINGS)	1E-5	1E-5
OPTIM	ADAMW_8BIT	ADAMW_8BIT
LR SCHEDULE	LINEAR	LINEAR
MODEL QUANTIZATION	4 BIT	4 BIT

5.2. MoE

Pretrained model is base model which could deal with general ARC-AGI tasks. After pretraining, we adopted the Mixture of Experts (MoE) methodology to enhance the model’s explanatory power.(Jordan & Jacobs, 1994) The MoE framework consists of a gating network and multiple expert networks. The gating network selects the top-k expert models, and the selected experts process the corresponding input. While most MoE applications in large language models (LLMs) operate on a token-by-token basis, we chose a simpler architecture for our implementation We chose the K-means clustering algorithm to classify training tasks and assign them to individual experts. (MacQueen, 1967) The clustering algorithm requires feature vectors extracted from each task. The features we used are:

- The ratio of input area to output area
- The ratio of color-changed cells to the total number of cells (set to 1 if input and output sizes differ)
- The ratio of the most frequent color to the total input area

For each feature, we computed the arithmetic mean across all input-output pairs in a task to obtain a representative value. Then, a logarithmic transformation was applied to suppress the effect of outliers. The area ratio feature was further discretized into three categories: > 1, $= 1$, and < 1. As a result, we classified 2,009 tasks into 6 clusters. The trained K-means clustering model was saved and reused during the inference stage.

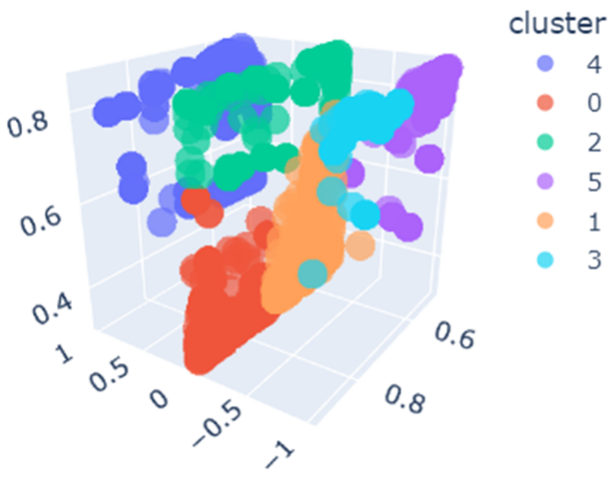

Figure 2. 3D Visualization of classified tasks in total dataset we have.

We selected a total of 800 tasks, with 188, 131, 120, 95, 83, and 183 tasks randomly sampled from each of the six clusters. During training, 500 examples were randomly sampled from each task, resulting in a total of $800 \times 500 = 400{,}000$ examples. These were grouped into 100,000 training instances, each containing 4 input-output pairs ($400{,}000/4 = 100{,}000$). Each training instance was then assigned to the corresponding expert based on its cluster label. We trained each expert starting from the base adapter using the same setup, but with a lower learning rate of 3×10^{-5}, to preserve the general inference capability learned during pretraining and to prevent overfitting. During inference, the gating network receives 4 input-output examples of the test task. It computes the representative feature vector and assigns a cluster label using the saved clustering model with a single-sample silhouette score. If the silhouette score exceeds 0.7, the corresponding expert performs the inference. Otherwise, the base model handles the task to avoid misrouting in ambiguous cases. From validation results, we observed that only the experts corresponding to cluster labels 0, 3, and 4 outperformed the base model. Therefore, the gating network routes the input to the base model if the assigned cluster label is not 0, 3, or 4.

6. Inference method

We followed the methodology proposed by Daniel Franzen *et al.* for performing inference based on the tokens generated by the model. Specifically, we employed a DFS-based probability-guided sampling algorithm to infer the correct answer for the given test input, using three provided examples. This method calculates the cumulative probability of each token candidate generated by the LLM and continues generation only for those tokens whose probability exceeds a certain threshold. This enables efficient acquisition of a diverse set of high-quality candidates. As ‘min_prob‘ decreases, there is a higher possibility of securing more candidates, but considering the trade-off between inference time and accuracy, we set it to 0.1. This DFS approach is applied simultaneously across multiple augmented perspectives. Given a task composed of three examples and one test input, we apply transformations from the D_8 symmetry group—such as rotations and reflections—to the input grids. Then, DFS is performed on each augmented task. After inference, the inverse of each transformation is applied to map the generated outputs back to the original task setting, yielding a variety of candidate solutions. In the final submitted model, due to time constraints, we limited augmentation to five operations: rotation by $0°$, rotation by $90°$, rotation by $180°$, reflection over the main diagonal, and reflection over the sub-diagonal. For the model used in test-time training (TTT), augmentation was conducted with only four variations, excluding the $90°$ rotation.

7. Contribution

Joon Seo Jang:

- Set up the Git-based development environment.
- Implemented basic data augmentation techniques.
- Introduced and integrated the inference strategy from the ARC Prize 2024 winning solution.

Soo Hyeong Kim:

- Adapted the initial fine-tuning code from the ARC Prize 2024 system to fit the scope of our custom task.
- Conducted visual inspection of all task data during augmentation to identify and classify invariant transformations.

- Proposed and implemented methods for more efficient fine-tuning of the model.
- Took the lead in writing the main body of the report.

Da Hyun Kang:

- Contributed to the development of the core methodology, including the adaptation of the training and inference strategy proposed by Daniel Franzen et al., the design of the learning rate scheduler, and the implementation of test-time training (TTT).
- Led the analysis of ensemble methods such as Borda voting and log-probability-based voting, as well as hyperparameter tuning for both training and inference stages.
- Implemented key components of the system, including initial fine-tuning of the base model (rank=256) using Unsloth and LoRA, optimization of prompt and token assignment, inference-time data augmentation, TTT, and ensemble logic.
- Authored Sections 5.1 and 6 of the report.

Su Bin Kim:

- Primarily contributed to the construction of additional training data, task clustering, and the design and implementation of the Mixture-of-Experts (MoE) training and gating mechanism.
- Retrained the rank=64 base model to accommodate multiple adapters within limited GPU VRAM.
- Generated additional task data, merged it with the augmented dataset, and built the clustering model and gating classifier for expert selection.

References

Bergsma, S., Dey, N., Gosal, G., Gray, G., Soboleva, D., and Hestness, J. Straight to zero: Why linearly decaying the learning rate to zero works best for llms, 2025. URL https://arxiv.org/abs/2502.15938.

Chollet, F. On the measure of intelligence. *CoRR*, abs/1911.01547, 2019. URL http://arxiv.org/abs/1911.01547.

Franzen, D., Disselhoff, J., and Hartmann, D. Solving the arc-agi is a matter of perspective. https://github.com/da-fr/arc-prize-2024, 2024. Kaggle ARC Prize 2024, First Place Solution.

Jordan, M. I. and Jacobs, R. A. Hierarchical mixtures of experts and the em algorithm. *Neural Computation*, 6(2): 181–214, 03 1994. ISSN 0899-7667. doi: 10.1162/neco.1994.6.2.181. URL https://doi.org/10.1162/neco.1994.6.2.181.

Li, W.-D., Hu, K., Larsen, C., Wu, Y., Alford, S., Woo, C., Dunn, S. M., Tang, H., Naim, M., Nguyen, D., Zheng, W.-L., Tavares, Z., Pu, Y., and Ellis, K. Combining induction and transduction for abstract reasoning, 2024. URL https://arxiv.org/abs/2411.02272.

MacQueen, J. Some methods for classification and analysis of multivariate observations. In *Proceedings of the Fifth Berkeley Symposium on Mathematical Statistics and Probability, Volume 1: Statistics*, pp. 281–297. University of California Press, 1967.

Meta AI. Llama 3 models, 2024. URL https://www.llama.com/models/llama-3/. Accessed: 2025-06-12.